

RDMA is Turing complete, we just did not know it yet! Links to an external site.

1 Paper summary

This paper argues that RDMA NICs can actually do much more than what they are usually used for. Normally, they are used for simple operations like reads, writes, or message passing, but more complex logic is still handled by the CPU.

The authors point out that this is a limitation, because many distributed systems want to offload work from the CPU, but doing things like hash-table lookups or linked-list traversal still requires CPU involvement. SmartNICs can solve this, but they are expensive and harder to deploy.

To address this, the paper proposes RedN. The key idea is to use something called self-modifying RDMA chains, where one operation can change the behavior of the next ones. This allows the NIC to implement things like conditionals and loops, without needing new hardware. It sounds a bit weird at first, but it's actually quite powerful.

They show that by combining RDMA verbs like CAS, WAIT, and ENABLE, it is possible to build something close to a general programming model on the NIC. Then they demonstrate this with examples like hash-table lookup and linked-list traversal, and also integrate it into Memcached to speed up key-value operations.

The results show significant improvements, like up to 2.6x lower latency compared to other RDMA-based systems, and even higher gains under contention. They also mention better resilience to crashes, since less work depends on the CPU. Overall, the main idea is that you don't necessarily need new hardware to get complex offloads, you just need to use existing features in a smarter way.

2 Contributions

The first contribution is the idea of using RDMA verbs to build more complex logic, like conditionals and loops. This is important because it shows that commodity RDMA NICs are more powerful than they seem, even if using them this way is not very straightforward.

The second contribution is the implementation of actual offloads, like hash-table lookup and linked-list traversal. This makes the work more concrete, since it shows that the approach is not just theoretical but can be used for real system tasks.

The third contribution is the integration with Memcached and the evaluation. The results show improvements in latency, especially under contention, and also better behavior in case of failures. This is useful because it connects the low-level idea to a real application, even if the evaluation is still somewhat limited.

3 Problems

One issue is that the programming model is quite low-level and not very intuitive. Developers need to think in terms of RDMA verbs, ordering, and self-modifying chains, which can be hard to understand and debug. So while the approach is powerful, it's probably not easy to use in practice.

Another concern is that the approach depends on specific RDMA features. The paper claims that it works on commodity hardware, but in reality different NICs may support different sets of verbs or behaviors. So portability could be a problem, even if it's not fully discussed.

A third point is that the evaluation focuses mostly on Memcached and a few examples. The results are good, but it's not clear how well this would scale to more complex systems

or larger applications. Also, building and maintaining this kind of offload logic could become difficult as systems grow, and the paper doesn't really go into that.